

P38 Parts Catalog

Manual arquitetural · UI First

Cada peça descreve: papel, caminho (pathway), conexões, filhas, regras e dados.
Gerado 15/08/2026 · sem prints · para comunicação com agentes Cursor.

Como ler

● Peça = página ou componente React

Caminho = sequência operacional (o que o utilizador/operador faz)

↑↓ Conexões = de quem recebe dados e para quem envia

◆ Função sync bloqueia ecrã; ■ roda em background/cron/trigger

■ Tabela + ■ FK = onde os dados repousam e como se ligam

Índice

1. **Início** — Home, Notificações / Agenda
2. **Dashboard** — Dashboard, Painel Gerente
3. **PDV** — PDVVendedor, PDV Supermercado, Auto-Atendimento
4. **Caixa** — PDVCaixa
5. **Vendas** — VendasGestao, ControleEntregas
6. **Produtos** — Produtos, HierarquiaPortal
7. **Compras** — Cotacoes, SugestoesCompra, PedidosCompra, PedidoCompraDetalhe, ConferenciaEntrada
8. **Estoque** — Armazenagem, ConferenciaEstoque, TabelaPrecosConsulta, ContagemExpress, MovimentosInventario, InterfaceSeparador
9. **Consumo Interno** — ConsumoInterno
10. **Financeiro** — FluxoCaixa, SuperAgefin, AprovacoesFinanceiras, FolhaPrevisao, ContasFinanceiras, VisaoFinanceira, CaixasAtivos, TurnosFechados, PlanejamentoFinanceiro, Budgets
11. **Satélites & PWA** — AnexoCompartilhado, DevolucaoTroca
12. **Relatórios** — Relatorios
13. **Configurações** — Configuracoes

Cadeias de fluxo (peças ligadas)

Venda completa (PDV → Caixa → Estoque → Financeiro)

1. PDVVendedor monta rascunho_pedido_venda + pedido_venda_item
2. PDVCaixa recebe rascunho → processarVendaCaixa
3. → pedido_venda (Financeiro OK) + movimentacao_estoque Saída
4. → lancamento_financeiro Receita + movimentos_caixa (dinheiro)
5. → trigger recalcula produto.estoque_atual
6. → ordem_separacao (se fluxo Completo) → InterfaceSeparador → ControleEntregas

Compra completa (Sugestão → PO → Financeiro → Recepção)

1. SugestoesCompra sugere qty → PedidosCompra cria pedido_compra
2. savePedidoCompraltem grava linhas → envio financeiro cria LF CMV
3. AprovacoesFinanceiras aprova → Aguardando Recepção
4. PedidoCompraDetalhe registra embarque → ConferenciaEntrada
5. movimentacao_estoque Entrada → trigger estoque
6. recalcular-conclusao-pedido-compra (async) confirma todos SKUs

Compromisso recorrente (AGEFIN → Fluxo)

1. conta_recorrente definida em SuperAgefin ou Planejamento
2. Cron gerarContasPrevistasRecorrentes → conta_prevista
3. Operador marca Pago → trigger sincronizarContaPrevia
4. → lancamento_financeiro → FluxoCaixa / saldo contas_financeiras

Devolução e vale troca

1. VendasGestao ou PDVCaixa identifica pedido_origem
2. DevolucaoTroca registra itens devolvidos
3. vale_compra gerado com valor_disponivel
4. Próximo PDVCaixa auto-preenche crédito do vale
5. processarVendaCaixa debita vale_compra

Conferência cega (código → estoque)

1. GestaoCodigosConferencia gera código (admin/Gerente)
2. ConferenciaVolumes ou Conferencialtem valida código
3. Operador conta volumes/itens
4. saveConferencialtem / finalização
5. movimentacao_estoque Entrada motivo Compra
6. trigger recalcula produto.estoque_atual

Início

Porta de entrada após login. Não processa vendas nem stock — agrega atalhos, alertas e tarefas do dia. É o hub de navegação rápida para quem acumula papéis.

Mapa hierárquico do módulo

Início

- ■ ■ ■ ● Home (/ · /Home)
- ■ ■ ■ quickActions.jsx
- ■ ■ ■ Central de Ações / tarefa
- ■ ■ ■ ● Notificações / Agenda (/Notificacoes)
- ■ ■ ■ agendaService.js

● Home

/ · /Home

pages/Home.jsx

Papel

Landing personalizada: quick actions filtradas por permissão do perfil; cards de alerta (pedidos pendentes, tarefas).

Árvore desta peça

Home

- ■ ■ ■ quickActions (atalhos por perfil)
- ■ ■ ■ cards alerta (pedidos, tarefas)
- ■ ■ ■ navigate → módulos operacionais

Lógica interna

Não grava transações. Lê usuario + perfil_de_acesso para decidir atalhos; React Query invalida alertas quando há mudanças noutros módulos.

Caminho (pathway)

1. Login → Home (mainPage em pages.config.js)
2. Utilizador toca atalho → navigate('/RotaDestino')
3. Alertas leem entidades em tempo real (React Query) sem gravar dados

Conexões

↑ recebe de: Auth/login

↑ recebe de: perfil_de_acesso (define atalhos visíveis)

↓ envia para: Qualquer módulo do menu

↓ envia para: Notificacoes (bottom nav mobile)

Filhas (componentes)

- quickActions.jsx — Lista de atalhos configuráveis (PDV, Compras, FluxoCaixa...)
- Central de Ações / tarefa — Tarefas com referencia_tipo/id polimórfica

Regras de negócio

▲ admin vê tudo; perfil sem permissões → menu mínimo (só Home)

Dados (Supabase)

■ usuario / tarefa / avisos_auto

■ tarefa.referencia_id → polimórfico

● Notificações / Agenda

/Notificacoes

pages/Notificacoes.jsx

Papel

Agenda operacional e lembretes; ícone 'Agenda' na bottom nav mobile.

Caminho (pathway)

1. Bottom nav → Notificacoes
2. Itens de agenda_item com status e responsável

Conexões

↑ recebe de: agendaService

↑ recebe de: módulos que criam tarefas

↓ envia para: Páginas referenciadas em tarefa.referencia_tipo

Filhas (componentes)

- agendaService.js — CRUD AgendItem + AgendaLogistica

Dados (Supabase)

■ agenda_item

■ responsavel_id → usuario.id (lógico)

Dashboard

Visão executiva por domínio. Lê snapshots KPI e entidades ao vivo — não é ponto de entrada operacional (venda/compra faz-se nos módulos).

Mapa hierárquico do módulo

Dashboard

- ■ ■ ■ ● Dashboard (/Dashboard)
- ■ ■ ■ Dashboard tabs (Geral/Vendas/...)
- ■ ■ ■ dashboardKpiSnapshotApi.js
- ■ ■ ■ ● Painel Gerente (/PainelGerente)
- ■ ■ ■ Tabela virtualizada + p38-mobile-line

● Dashboard

/Dashboard

paiol/pages/Dashboard

Papel

Painel com abas Geral · Vendas · Compras · Estoque · Financeiro; gráficos e KPIs.

Caminho (pathway)

1. Menu Dashboard → carrega KPIs do dia/mês
2. Aba escolhida filtra domínio (vendas, compras...)
3. Drill-down navega para módulo fonte (ex.: Produtos, PedidosCompra)

Conexões

↑ recebe de: dashboard_kpi_diario/mensal (snapshots)
↑ recebe de: pedido_venda, pedido_compra, produto, lancamento_financeiro
↓ envia para: Módulos operacionais via links contextuais

Filhas (componentes)

- Dashboard tabs (Geral/Vendas/...) — Cada aba agrega queries por domínio
- dashboardKpiSnapshotApi.js — Leitura RPC snapshots Supabase

Regras de negócio

▲ KPIs fechados por cron fechar-dashboard-kpi (ontem)

Funções (lógica server)

◆ fechar-dashboard-kpi [■ background] — Cron: materializa KPI do dia anterior

Background / performance

■ job_fechar_dashboard_kpi_ontem via pg_cron

Dados (Supabase)

■ dashboard_kpi_diario / dashboard_kpi_mensal

● Painel Gerente

/PainelGerente

pages/PainelGerente.jsx

Papel

Visão de vendas para gestor: ranking, metas, pedidos do período.

Caminho (pathway)

1. Menu Vendas → Painel Gerente
2. Filtro período → lista pedido_venda

Conexões

- ↑ recebe de: pedido_venda
- ↑ recebe de: rascunho_pedido_venda
- ↑ recebe de: usuario (vendedor)
- ↓ envia para: VendasGestao (detalhe pedido)
- ↓ envia para: PDVVendedor (novo pedido)

Filhas (componentes)

- Tabela virtualizada + p38-mobile-line — Lista responsiva mobile/desktop

Dados (Supabase)

- pedido_venda
- cliente_id → terceiro.id (FK)

PDV

Ponto de venda — onde nasce o rascunho. Três portas (vendedor, supermercado, totem) convergem no mesmo funil: rascunho → caixa → pedido_venda.

Mapa hierárquico do módulo

```

PDV
  ■■■● PDVVendedor (/PDVVendedor)
  ■■■■ BarcodeScanner
  ■■■■ ComprovantePreVenda
  ■■■■ LostSalesForm
  ■■■■ ProductUnitSelectorDialog
  ■■■● PDV Supermercado (/PDV?mode=supermercado)
  ■■■■ SimuladorCartaoSheet
  ■■■● Auto-Atendimento (/AutoAtendimento)
  ■■■■ AutoShop
  ■■■■ AutoPayment
  ■■■■ ProductDetailDialog
  
```

● PDVVendedor

/PDVVendedor

components/vendas/PDVVendedor.jsx

Papel

Venda assistida: monta carrinho, identifica cliente, gera orçamento ou envia ao caixa.

Árvore desta peça

```

PDVVendedor
  ■■■■ BarcodeScanner → produto
  ■■■■ ProductUnitSelectorDialog → unidade/qty_base
  ■■■■ savePedidoVendaItem → rascunho_pedido_venda
  ■■■■ ComprovantePreVenda (pré-caixa)
  ■■■■ LostSalesForm → venda_perdida
  
```

Lógica interna

O rascunho é a peça intermédia: todas as linhas vivem em rascunho_pedido_venda até o caixa chamar processarVendaCaixa. O vendedor nunca baixa estoque nem cria LF — isso é responsabilidade exclusiva do caixa (transação atômica).

Caminho (pathway)

1. Operador abre turno (opcional no vendedor) e escaneia/busca produto
2. ProductUnitSelectorDialog resolve unidade comercial → quantidade_base
3. savePedidoVendaItem grava linhas em rascunho_pedido_venda
4. 'Enviar ao caixa' → status rascunho aguarda PDVCaixa
5. Caixa chama processarVendaCaixa → pedido_venda definitivo

Conexões

↑ recebe de: Produtos (catálogo, preço, estoque)
 ↑ recebe de: Terceiros (cliente)
 ↑ recebe de: tabela_preco do vendedor
 ↓ envia para: PDVCaixa (rascunho)
 ↓ envia para: VendasGestao (consulta)
 ↓ envia para: InterfaceSeparador (se fluxo Completo)

Filhas (componentes)

- **BarcodeScanner** — Resolve código de barras → produto.id via matching
- **ComprovantePreVenda** — Pré-visualização antes do caixa (não baixa estoque)
- **LostSalesForm** — Regista venda_perdida quando cliente desiste
- **ProductUnitSelectorDialog** — Escolhe embalagem/unidade; calcula fator_conversao → qty_base

Regras de negócio

- ▲ Bloqueia add se estoque_atual < qty_base, salvo vender_sem_estoque ou permitir_venda_estoque_negativo
- ▲ Rascunho é a 'peça-mãe' até o caixa converter — não cria pedido_venda ainda

Funções (lógica server)

- ◆ **savePedidoVendaItem** [sync (bloqueia UI)] — Espelha linhas no JSON legado + tabela canónica
- ◆ **gerarNumeroSequencial** [sync (bloqueia UI)] — PV-* para orçamentos

Dados (Supabase)

- **rascunho_pedido_venda**
- cliente_id → terceiro.id

- vendedor_id → usuario.id
- pedido_venda_final_id → pedido_venda.id (após conversão)
- **pedido_venda_item**
 - pedido_venda_id → pedido_venda.id (lógico)
 - produto_id → produto.id (lógico)

● PDV Supermercado

/PDV?mode=supermercado

components/vendas/PDVSupermercado.jsx

Papel

Checkout rápido: menos passos, pode finalizar venda no próprio ecrã.

Caminho (pathway)

1. Scan contínuo → pagamento → processarVendaCaixa (se caixa embutido)

Conexões

↑ recebe de: produto

↑ recebe de: configuracoes_venda

↓ envia para: pedido_venda

↓ envia para: movimentacao_estoque (saída)

Filhas (componentes)

● **SimuladorCartaoSheet** — Simula taxa maquininha antes de confirmar

Regras de negócio

▲ Mesmas regras de estoque do vendedor

Funções (lógica server)

◆ **processarVendaCaixa** [sync (bloqueia UI)] — RPC atómica — ver Caixa

Dados (Supabase)

■ **pedido_venda**

■ cliente_id → terceiro.id (FK DB)

● Auto-Atendimento

/AutoAtendimento

pages/AutoAtendimento.jsx

Papel

Totem fullscreen: jornada guiada sem operador.

Caminho (pathway)

1. AutoHome → AutoIdentification (CPF/telefone)
2. → AutoShop (grade produtos config_auto_atendimento)
3. → AutoPayment → processarVendaCaixa

Conexões

- ↑ recebe de: config_auto_atendimento
- ↑ recebe de: produto (ativos na vitrine)
- ↓ envia para: pedido_venda
- ↓ envia para: venda_perdida (abandono)

Filhas (componentes)

- **AutoShop** — Grelha de produtos com fotos e destaques
- **AutoPayment** — Seleção forma pagamento + confirmação
- **ProductDetailDialog** — Detalhe embalagem no totem

Regras de negócio

- ▲ Fullscreen; sem sidebar; timeout pode registrar venda_perdida

Funções (lógica server)

- ◆ processarVendaCaixa [sync (bloqueia UI)] — Mesmo RPC do caixa

Dados (Supabase)

- config_auto_atendimento
- venda_perdida
- produto_id → produto.id

Caixa

Onde o rascunho vira venda real. É o único ponto que chama `processarVendaCaixa` — baixa estoque, cria lançamentos financeiros e liga ao `turno_caixa`.

Mapa hierárquico do módulo

```

Caixa
  ■■■● PDVCaixa (/PDVCaixa)
  ■■■■ ProcessarVendasView
  ■■■■ ConfirmarPagamentoDialog
  ■■■■ VendasTurnoDialog
  ■■■■ SeletorCaixaPDV
  ■■■■ VisualizadorCaixa
  
```

● PDVCaixa

/PDVCaixa

`components/vendas/PDVCaixa.jsx`

Papel

Recebimento, sangria, reforço, cupom, devolução. Centro nervoso do PDV.

Árvore desta peça

```

PDVCaixa
  ■■■■ SeletorCaixaPDV → turno_caixa
  ■■■■ ProcessarVendasView → rascunhos (poll)
  ■■■■ ConfirmarPagamentoDialog
  ■■■■ processarVendaCaixa (RPC)
  ■■■■ VendasTurnoDialog (espelho)
  ■■■■ sangria/reforço → movimentos_caixa
  ■■■■ imprimirCupomTermico
  
```

Lógica interna

`processarVendaCaixa` é a única porta que converte rascunho em venda real numa transação: valida idempotência (409 se já processado), cria `pedido_venda`, gera saída em `movimentacao_estoque`, cria `lançamento_financeiro` por forma de pagamento (dinheiro atualiza saldo conta; cartão exige maquininha; fiado gera LF Em Aberto). O trigger de estoque corre depois, sem bloquear o RPC.

Caminho (pathway)

1. `SeletorCaixaPDV`: operador escolhe `conta_caixa_pdv` + abre `turno_caixa`
2. `ProcessarVendasView`: lista rascunhos do turno (polling live)
3. `ConfirmarPagamentoDialog`: formas pagamento, maquininha, vale troca
4. `processarVendaCaixa` (RPC): rascunho→pedido, saída estoque, LF receita
5. `imprimirCupomTermico` → comprovante
6. `Sangria/Reforço` → `movimentos_caixa` + `lançamento_financeiro`

Conexões

```

↑ recebe de: PDVVendedor (rascunho_pedido_venda)
↑ recebe de: DevolucaoTroca / vale_compra (crédito cliente)
↑ recebe de: FormasDePagamento + Maquininha (regras cartão)
↓ envia para: turno_caixa + movimentos_caixa
↓ envia para: lançamento_financeiro (receitas, sangrias)
↓ envia para: movimentacao_estoque (saída venda)
↓ envia para: ordem_separacao (se fluxo Completo)
↓ envia para: CaixasAtivos / TurnosFechados (monitoramento)
  
```

Filhas (componentes)

- **ProcessarVendasView** — Fila de rascunhos; botão processar abre `ConfirmarPagamentoDialog`
- **ConfirmarPagamentoDialog** — Valida cobertura R\$0,01; exige maquininha em cartão
- **VendasTurnoDialog** — Espelho vendas do turno (status elegíveis)
- **SeletorCaixaPDV** — Vincula operador à `conta_caixa_pdv_id`
- **VisualizadorCaixa** — Reutilizado em `CaixasAtivos/TurnosFechados`

Regras de negócio

- ▲ Sem turno = somente leitura
- ▲ Anti-duplo-clique: `processandoVenda` trava reentrada
- ▲ Cartão: líquido = bruto × (1−taxa/100); vencimento dias úteis
- ▲ Fiado cria LF Receita Em Aberto tag FIADO

Funções (lógica server)

- ◆ `processarVendaCaixa` [sync (bloqueia UI)] — RPC transacional; idempotente (409 se já processado)
- ◆ `imprimirCupomTermico` [sync (bloqueia UI)] — Template `ComprovanteTemplate` + `DadosEmpresa`

◆ **gerenciarPin** [sync (bloqueia UI)] — PIN 6 dígitos (opcional via env)

Background / performance

- Trigger sincronizarEstoquePorMovimentacao após cada movimentacao_estoque
- Polling rascunhos (CAIXA_LIVE_POLL_MS) — UI não bloqueia

Dados (Supabase)

- **turno_caixa**
 - conta_caixa_pdv_id → contas_financeiras.id
 - operador_id → usuario.id
- **movimentos_caixa**
 - conta_id → contas_financeiras.id (FK RESTRICT)
 - turno_caixa_id → turno_caixa.id (FK SET NULL)
- **lancamento_financeiro**
 - conta_financeira_id → contas_financeiras.id (FK)
 - referencia_id → pedido_venda | devolucao_troca (polimórfico)
- **pedido_venda**
 - cliente_id → terceiro.id (FK)

Vendas

Pós-venda: consulta, alteração, entregas e perdas. Não substitui o PDV para venda nova.

Mapa hierárquico do módulo

- Vendas
 - ● VendasGestao (/VendasGestao)
 - ■■■■ DetalhesPedidoVenda
 - ■■■■ AlterarPagamentoDialog
 - ■■■■ ValesTrocaTab
 - ■■■■ ConsultaVendasCaixa
 - ● ControleEntregas (/ControleEntregas)
 - ■■■■ LiberacaoEntrega

● VendasGestao

/VendasGestao

pages/VendasGestao.jsx

Papel

Lista virtualizada de pedidos/orçamentos com filtros e ações em lote.

Caminho (pathway)

1. Filtro período/status → query pedido_venda
2. Clique linha → DetalhesPedidoVenda (sheet)
3. Alterar pagamento / reimprimir / devolver

Conexões

- ↑ recebe de: PDVCaixa (pedidos processados)
- ↑ recebe de: PDVVendedor (orçamentos)
- ↓ envia para: DevolucaoTroca
- ↓ envia para: ControleEntregas
- ↓ envia para: PDVCaixa (retorno edição)

Filhas (componentes)

- **DetalhesPedidoVenda** — Sheet com itens, pagamentos, histórico
- **AlterarPagamentoDialog** — Ajusta pagamentos pós-venda (regras financeiras)
- **ValesTrocaTab** — Lista vales do cliente
- **ConsultaVendasCaixa** — Visão caixa integrada na gestão

Regras de negócio

- ▲ Lista virtualizada para performance em alto volume

Funções (lógica server)

- ◆ **savePedidoVendaItem** [sync (bloqueia UI)] — Edição linhas em pedidos abertos

Dados (Supabase)

- **pedido_venda / pedido_venda_item**
 - cliente_id → terceiro.id (FK)
 - pedido_venda_item.pedido_venda_id → pedido_venda.id

● ControleEntregas

/ControleEntregas

pages/ControleEntregas.jsx → LiberacaoEntrega

Papel

Agenda e liberação de entregas ao cliente.

Caminho (pathway)

1. Pedido Em Rota → agenda_logistica → motorista → protocolo_entrega

Conexões

↑ recebe de: pedido_venda (status entrega)
↑ recebe de: PDV fluxo Completo + ordem_separacao
↓ envia para: protocolo_entrega
↓ envia para: Expedicao

Filhas (componentes)

● LiberacaoEntrega — Confirma saída e atualiza status pedido

Dados (Supabase)

■ agenda_logistica
■ pedido_venda_id → pedido_venda.id (FK CASCADE)
■ cliente_id → terceiro.id (FK RESTRICT)

Produtos

Catálogo-mãe do sistema. Quase todos os módulos leem produto; alterações aqui propagam para PDV, compras, estoque e margem.

Mapa hierárquico do módulo

```

Produtos
■ ■ ■ ■ ● Produtos (/Produtos)
■ ■ ■ ■ TreeGrid
■ ■ ■ ■ MobileHierarquica
■ ■ ■ ■ ProdutoFormCompleto
■ ■ ■ ■ ProdutoFAB
■ ■ ■ ■ ● HierarquiaPortal (/HierarquiaPortal)
■ ■ ■ ■ PortalTreeGrid
■ ■ ■ ■ CadastroProdutoV2Form
  
```

● Produtos

/Produtos

pages/Produtos.jsx

Papel

CRUD catálogo, precificação, ABCD/IEP, relatórios embutidos (?relatorioVendas/Estoque=1).

Árvore desta peça

```

Produtos
■ ■ ■ ■ TreeGrid (desktop) / MobileHierarquica (mobile)
■ ■ ■ ■ termômetro margem≥30% markup≥40%
■ ■ ■ ■ ProdutoFormCompleto
■ ■ ■ ■ ProdutoFAB → ações rápidas
■ ■ ■ ■ jobs: calcularIEP / atualizarMetasEstoque
  
```

Lógica interna

Produto é hub de leitura para quase todo o sistema. Alterar preço ou estoque aqui propaga para PDV, sugestão de compra e margem. O termômetro mobile calcula margem/markup ao vivo sem gravar; o job calcularIEP grava curva ABCD só se ABCD_JOB_NOTURNO estiver ativo.

Caminho (pathway)

1. TreeGrid (desktop) ou MobileHierarquica (mobile) lista SKUs
2. ProdutoFAB → ProdutoFormCompleto (cadastro completo)
3. Painel precificação mobile: semáforo margem/markup em tempo real
4. Job noturno calcularIEP grava curva ABCD (se ABCD_JOB_NOTURNO)

Conexões

```

↑ recebe de: ImportacaoProdutos
↑ recebe de: HierarquiaPortal (grade)
↑ recebe de: pedido_compra (custos)
↓ envia para: PDV (preço, estoque)
↓ envia para: SugestoesCompra (metas)
↓ envia para: RelatorioMargem
  
```

Filhas (componentes)

- **TreeGrid** — Grelha hierárquica h1..h5 com windowing performance
- **MobileHierarquica** — Precificação mobile + termômetro margem≥30% markup≥40%
- **ProdutoFormCompleto** — Formulário completo: unidades, custos, fornecedor
- **ProdutoFAB** — Ações rápidas: novo, importar, relatório

Regras de negócio

- ▲ Margem = (preço-custo)/preço×100; markup alvo global 40% (Preço Justo)
- ▲ ABCD pareto ao vivo; iep_trava_manual preserva classe gravada
- ▲ estoque_atual recalculado por trigger em movimentacao_estoque

Funções (lógica server)

- ◆ **calcularIEP** [■ background] — Batch 50 SKUs; preparar→classificar→gravar
- ◆ **atualizarMetasEstoque** [■ background] — Ponto de pedido; janela 60d
- ◆ **importarProdutos** [■ background] — Bulk + ImportacaoLog undo

Dados (Supabase)

```

■ produto
■ categoria_id → categoria_produto.id
■ fornecedor_padrao_id → terceiro.id
  
```


- **movimentacao_estoque**
 - produto_id → produto.id (FK RESTRICT)

● HierarquiaPortal

/HierarquiaPortal

*hierarquia-portal/**

Papel

Grade de compra (linha→produto_compra→eixos) + cadastro v2 cerâmica.

Caminho (pathway)

1. PortalTreeGrid navega grade → CadastroProdutoV2Form cria SKU portal

Conexões

↑ recebe de: modelo_linha / modelo_produto_compra (laboratório)

↓ envia para: produto (vínculo portal_catalog)

↓ envia para: SugestoesCompra

Filhas (componentes)

- **PortalTreeGrid** — Árvore linha/eixo com reserva portal
- **CadastroProdutoV2Form** — Wizard cadastro com grade

Regras de negócio

▲ Grade: linha_compra → produto_compra → eixo_valor (FKs DB na grade)

Dados (Supabase)

- **portal_catalog / linha_compra / eixo_valor**
- produto_compra.linha_id → linha_compra.id (FK)

Compras

Ciclo procure-to-receive: sugestão → cotação → PO → aprovação financeira → embarque → conferência → entrada estoque.

Mapa hierárquico do módulo

```

Compras
  ■■■● Cotacoes (/Cotacoes)
  ■■■■ CotacaoExpressHub
  ■■■■ ImportadorCotacaoPDF
  ■■■■● SugestoesCompra (/SugestoesCompra)
  ■■■■ SugestaoCompraTreeGrid
  ■■■■ SugestaoCompraMobileCatalog
  ■■■■● PedidosCompra (/PedidosCompra)
  ■■■■ ListaPedidosCompra
  ■■■■ ConsultaComprasPedidos
  ■■■■ ImportadorNotaFiscal
  ■■■■ ActionMenuComprasV2
  ■■■■● PedidoCompraDetalhe (/PedidoCompraDetalhe?id=)
  ■■■■ InformarEmbarque
  ■■■■ RecepcionarEmbarque
  ■■■■ PedidoCompraFAB
  ■■■■● ConferenciaEntrada (/ConferenciaEntrada)
  ■■■■ GestaoCodigosConferencia
  ■■■■ PainelConferencias
  
```

● Cotacoes

/Cotacoes

components/compras/CotacoesManager.jsx

Papel

Cotação com fornecedores antes de formalizar PO.

Árvore desta peça

```

Cotacoes
  ■■■■ CotacaoExpressHub
  ■■■■ CotacaoExpressMontagem / Disputa
  ■■■■ ImportadorCotacaoPDF
  
```

Lógica interna

Cotação não movimenta estoque nem financeiro — só compara preços. PO nasce depois em PedidosCompra.

Caminho (pathway)

1. Monta lista produtos + fornecedores convidados
2. Fornecedor responde (ou import PDF)
3. Escolhe vencedor → gera pedido_compra rascunho

Conexões

↑ recebe de: produto
 ↑ recebe de: terceiro (fornecedor)
 ↓ envia para: PedidosCompra (novo PO)

Filhas (componentes)

- **CotacaoExpressHub** — Hub express de cotações abertas
- **ImportadorCotacaoPDF** — OCR/import PDF resposta fornecedor

Funções (lógica server)

- ◆ **gerarNumeroSequencial** [sync (bloqueia UI)] — Número cotação

Dados (Supabase)

- **cotacao**
- **fornecedores/respostas** em JSON

● SugestoesCompra

/SugestoesCompra

components/compras/SugestaoCompra.jsx

Papel

Smart Supply: quanto comprar com base em metas, ABCD e vendas.

Caminho (pathway)

1. Filtro curva → árvore produtos → qty sugerida → gera rascunho PO

Conexões

- ↑ recebe de: produto (estoque, metas)
- ↑ recebe de: pedido_venda (demanda)
- ↓ envia para: PedidosCompra (novo PO)
- ↓ envia para: HierarquiaPortalEntry

Filhas (componentes)

- **SugestaoCompraTreeGrid** — Desktop: árvore com qty editável
- **SugestaoCompraMobileCatalog** — Mobile: catálogo compacto

Regras de negócio

- ▲ **Filtro ABCD; velocidade de venda editável**

Funções (lógica server)

- ◆ **atualizarMetasEstoque** [■ background] — Job background

Dados (Supabase)

- **produto / pedido_compra**

● PedidosCompra

/PedidosCompra

pages/PedidosCompra.jsx

Papel

Lista e gestão de POs; ponto de envio ao financeiro.

Caminho (pathway)

1. Criar PO → savePedidoCompraltem (linhas canônicas)
2. Enviar financeiro → LF is_custo_mercadoria:true
3. AprovacoesFinanceiras aprova → Aguardando Recepção
4. Embarque → conferência → recepção → estoque

Conexões

↑ recebe de: SugestoesCompra
↑ recebe de: Cotacoes
↑ recebe de: ImportadorNotaFiscal
↓ envia para: AprovacoesFinanceiras
↓ envia para: PedidoCompraDetalhe
↓ envia para: ConferenciaEntrada
↓ envia para: lancamento_financeiro

Filhas (componentes)

- **ListaPedidosCompra** — Tabela principal com status workflow
- **ConsultaComprasPedidos** — Consulta avançada embarques pendentes
- **ImportadorNotaFiscal** — PDF NF → linhas PO
- **ActionMenuComprasV2** — Ações: aprovar, PDF, financeiro

Regras de negócio

- ▲ isLocked após envio financeiro (não edita sem reabertura)
- ▲ Trigger: status Aprovado → aprovacao_financeira automática
- ▲ Reabertura bloqueada se manifesto_entrada_id preenchido

Funções (lógica server)

- ◆ **savePedidoCompraltem** [sync (bloqueia UI)] — qty_base, custo líquido, total PO
- ◆ **recalcular-conclusao-pedido-compra** [■ background] — Estoque pós-recepção

Dados (Supabase)

- **pedido_compra**
 - fornecedor_id → terceiro.id (FK RESTRICT)
 - conta_pagamento_id → contas_financeiras.id (FK SET NULL)
- **pedido_compra_item**
 - pedido_compra_id → pedido_compra.id (lógico)
 - produto_id → produto.id (lógico)

● PedidoCompraDetalhe

/PedidoCompraDetalhe?id=

components/compras/PedidoCompraForm.jsx

Papel

Fullscreen: edição completa PO, embarques, anexos, relatórios.

Caminho (pathway)

1. Lista PO → detalhe → abas itens/embarques/financeiro/anexos

Conexões

- ↑ recebe de: PedidosCompra
- ↓ envia para: embarque
- ↓ envia para: ConferenciaEntrada
- ↓ envia para: Financeiro

Filhas (componentes)

- **InformarEmbarque** — Regista embarque + saveEmbarqueItem
- **RecepcionarEmbarque** — Confirma recepção física
- **PedidoCompraFAB** — PDF pedido, precificação, pendências

Regras de negócio

- ▲ PIN save opcional em Aguardando Aprovação Financeira

Funções (lógica server)

- ◆ **saveEmbarqueItem** [sync (bloqueia UI)] — qty embarcada/recebida por linha
- ◆ **uploadAnexoDrive** [sync (bloqueia UI)] — Anexo NF/contrato no Drive

Dados (Supabase)

- **embarque / embarque_item**
- **embarque.pedido_compra_id** → **pedido_compra.id** (FK CASCADE)
- **embarque_item.pedido_compra_item_id** → **pedido_compra_item.id**

● ConferenciaEntrada

/ConferenciaEntrada

PainelConferencias + GestaoCodigosConferencia

Papel

Recepção cega: código → volumes ou itens → entrada estoque.

Caminho (pathway)

1. Gerente gera código (generateConferenceCode)
2. Conferente escaneia em ConferenciaVolumes ou Conferencialtems
3. validateConferenceCode → status Em Uso
4. Finalizar → movimentacao_estoque Entrada motivo Compra

Conexões

↑ recebe de: supermanifesto / manifesto_entrada

↑ recebe de: embarque

↓ envia para: movimentacao_estoque

↓ envia para: produto.estoque_atual (trigger)

Filhas (componentes)

- GestaoCodigosConferencia — Admin: gera/expira códigos
- PainelConferencias — Lista conferências ativas

Regras de negócio

- ▲ Código 8 chars; só admin/Gerente gera
- ▲ Volumes: permite divergência; Itens: exige lote se controla_lote

Funções (lógica server)

- ◆ generateConferenceCode [sync (bloqueia UI)] —
- ◆ validateConferenceCode [sync (bloqueia UI)] —
- ◆ saveConferencialtem [sync (bloqueia UI)] — divergencia_base = contada-sistema

Dados (Supabase)

- supermanifesto / manifesto_entrada / conferencia_compra

Estoque

Movimentação física e contagem. Toda entrada/saída passa por movimentacao_estoque.

Mapa hierárquico do módulo

```

Estoque
  ■■■■ ● Armazenagem (/Armazenagem)
  ■ ■■■■ FilaSeparacao
  ■ ■■■■ MovimentacaoEstoqueForm
  ■■■■ ● ConferenciaEstoque (/ConferenciaEstoque · /ConferenciaEditor?id=)
  ■ ■■■■ NovaConferenciaDialog
  ■ ■■■■ ConferenciaEditor
  ■■■■ ● TabelaPrecosConsulta (/TabelaPrecosConsulta)
  ■ ■■■■ OrcamentoSheet
  ■■■■ ● ContagemExpress (/ContagemExpress)
  ■ ■■■■ ContagemExpressCarrinho
  ■ ■■■■ ContagemExpressPainelContagem
  ■■■■ ● MovimentosInventario (/MovimentosInventario)
  ■ ■■■■ productMatchingUtils
  ■■■■ ● InterfaceSeparador (/InterfaceSeparador)

```

● Armazenagem

/Armazenagem

pages/Armazenagem.jsx

Papel

Localização física: corredor, prateleira, área de picking.

Árvore desta peça

```

Armazenagem
  ■■■■ FilaSeparacao
  ■■■■ MovimentacaoEstoqueForm
  ■■■■ HistoricoMovimentacoes

```

Lógica interna

Altera area_id do produto e pode gerar movimentação de transferência entre áreas.

Caminho (pathway)

1. Busca SKU → edita localização → opcional movimento transferência

Conexões

```

↑ recebe de: produto
↑ recebe de: area
↓ envia para: movimentacao_estoque (transferência)
↓ envia para: InterfaceSeparador

```

Filhas (componentes)

- **FilaSeparacao** — Pedidos aguardando separação física
- **MovimentacaoEstoqueForm** — Form movimento manual

Dados (Supabase)

- **produto / area**
- **produto.area_id** → **area.id**

● ConferenciaEstoque

/ConferenciaEstoque · /ConferenciaEditor?id=

estoque/auditoria/*

Papel

Contagem formal com sessão, ajuste e auditoria.

Árvore desta peça

ConferenciaEstoque

■■■■ NovaConferenciaDialog

■■■■ ConferenciaEditor → saveConferenciaItem

Lógica interna

Diferente da Contagem Express: aqui a conferência é uma entidade com ciclo de vida (aberta → contada → ajuste aplicado).

O ajuste gera movimentacao_estoque compensatória.

Caminho (pathway)

1. Nova conferência (nome, responsável)
2. Contagem item a item no Editor
3. saveConferencialtem grava divergência
4. Aplicar ajuste → movimentacao_estoque

Conexões

↑ recebe de: produto (estoque sistema)

↓ envia para: movimentacao_estoque

↓ envia para: AuditoriaEstoque

Filhas (componentes)

● **NovaConferenciaDialog** — Cria sessão conferencia_estoque

● **ConferenciaEditor** — UI contagem por linha

Regras de negócio

▲ **divergencia_base** = qty contada – qty sistema

Funções (lógica server)

◆ **saveConferencialtem** [sync (bloqueia UI)] — Linhas canônicas

Dados (Supabase)

■ **conferencia_estoque**

■ **responsavel_id** → usuario.id

● **TabelaPrecosConsulta**

/TabelaPrecosConsulta

pages/TabelaPrecosConsulta.jsx

Papel

Consulta rápida de preço na loja (operador de chão).

Árvore desta peça

- TabelaPrecosConsulta
 - OrcamentoSheet
 - SimuladorCartaoSheet

Lógica interna

Somente leitura de tabela_preco + produto; pode gerar orçamento PDF sem venda.

Caminho (pathway)

1. Scan/busca → mostra preço por tabela → opcional orçamento

Conexões

- ↑ recebe de: tabela_preco
- ↑ recebe de: produto
- ↑ recebe de: dados_empresa
- ↓ envia para: PDVVendedor (orçamento)
- ↓ envia para: venda_perdida (se desiste)

Filhas (componentes)

- **OrcamentoSheet** — Gera orçamento rápido para cliente

Dados (Supabase)

- **tabela_preco / produto**

● ContagemExpress

/ContagemExpress

estoque/contagem-express/*

Papel

Contagem rápida com carrinho; confirma com PIN.

Caminho (pathway)

1. Scan produto → carrinho qty → PIN → conferencia_estoque

Conexões

↑ recebe de: produto

↓ envia para: conferencia_estoque → ajuste movimentacao

Filhas (componentes)

- ContagemExpressCarrinho — Lista itens contados
- ContagemExpressPainelContagem — Input qty + scan

Regras de negócio

▲ PIN confirmação (se operacao auth ativo)

Funções (lógica server)

◆ saveConferencialtem [sync (bloqueia UI)] —

Dados (Supabase)

- conferencia_estoque
- itens[].produto_id → produto

● MovimentosInventario

/MovimentosInventario

pages/MovimentosInventario.jsx

Papel

Ajuste manual: entrada, saída, transferência entre áreas.

Caminho (pathway)

1. Busca produto → tipo movimento → grava movimentacao_estoque → trigger recalcula

Conexões

↑ recebe de: produto

↑ recebe de: area

↓ envia para: movimentacao_estoque → produto.estoque_atual

Filhas (componentes)

● productMatchingUtils — Resolve busca por código/nome

Regras de negócio

▲ Negativo permitido desde mig.038

Background / performance

■ trigger recalcular_estoque_produto

Dados (Supabase)

■ movimentacao_estoque

■ produto_id → produto.id (FK RESTRICT)

● InterfaceSeparador

/InterfaceSeparador

pages/InterfaceSeparador.jsx

Papel

Fila de separação: pedidos prontos para expedição.

Caminho (pathway)

1. ordem_separacao Pendente → separador confirma itens → Em Separação/Concluído

Conexões

↑ recebe de: processarVendaCaixa (ordem_separacao se fluxo Completo)

↓ envia para: ControleEntregas

↓ envia para: Expedicao

Dados (Supabase)

■ ordem_separacao

■ pedido_venda_id → pedido_venda.id (lógico)

Consumo Interno

Baixa de mercadoria para uso interno (refeitório, manutenção). Gera movimentação de saída.

Mapa hierárquico do módulo



● ConsumoInterno

/ConsumoInterno

pages/ConsumoInterno.jsx

Papel

Registro de consumo com destinação, responsável e aprovação.

Caminho (pathway)

- 1. 1. Selecciona destinação + responsável (lookups)
- 2. 2. Adiciona itens produto × qty
- 3. 3. Confirma → movimentacao_estoque Saída + consumo_interno
- 4. 4. Pode vincular turno_caixa se PDV aberto

Conexões

- ↑ recebe de: produto
- ↑ recebe de: destinacao_consumo_interno
- ↑ recebe de: responsavel_consumo_interno
- ↓ envia para: movimentacao_estoque
- ↓ envia para: CaixasAtivos (monitor)

Filhas (componentes)

- RelatorioConsumoInterno — Relatório por período/destinação

Regras de negócio

- ▲ gerarNumeroSequencial CI-*

Funções (lógica server)

- ◆ gerarNumeroSequencial [sync (bloqueia UI)] — CI-*

Dados (Supabase)

- consumo_interno
 - turno_caixa_id → turno_caixa.id (lógico)
 - itens[].produto_id → produto.id

Financeiro

Dinheiro do negócio: lançamentos, contas, recorrência, aprovações. Gate opcional VITE_FINANCEIRO_GATE_PASSWORD (15 min).

Mapa hierárquico do módulo

```

Financeiro
  ■■■● FluxoCaixa (/FluxoCaixa)
  ■■■■ ListaLancamentos
  ■■■■ FiltrosFluxoCaixa
  ■■■■ ConciliacaoBancaria
  ■■■■ NovoLancamentoDialog
  ■■■■ SuperAgefin (/SuperAgefin)
  ■■■■ SuperAgefinConsultaDrawer
  ■■■■ SuperAgefinConsultaOrganizer
  ■■■■ AprovacoesFinanceiras (/AprovacoesFinanceiras)
  ■■■■ aprovarPedidoCompraFinanceiro.js
  ■■■■ FolhaPrevisao (/FolhaPrevisao)
  ■■■■ FolhaSimulacao
  ■■■■ ContasFinanceiras (/ContasFinanceiras)
  ■■■■ ConciliacaoBancaria
  ■■■■ VisaoFinanceira (/VisaoFinanceira)
  ■■■■ planoFinanceiroConsolidado.js
  ■■■■ CaixasAtivos (/CaixasAtivos)
  ■■■■ VisualizadorCaixa
  ■■■■ TurnosFechados (/TurnosFechados)
  ■■■■ CaixaValorDisplay
  ■■■■ PlanejamentoFinanceiro (/PlanejamentoFinanceiro)
  ■■■■ ContasFixasTab
  ■■■■ Budgets (/Budgets)
  ■■■■ BudgetPlanoCompleto
  
```

● FluxoCaixa

/FluxoCaixa

financeiro/ExecucaoOrcamentaria.jsx

Papel

Execução orçamentária: lista lançamentos, filtros, conciliação, novo LF.

Caminho (pathway)

1. FiltrosFluxoCaixa → ListaLancamentos
2. NovoLancamentoDialog → grava lancamento_financeiro
3. ConciliacaoBancaria → agrupa LF por extrato
4. ?aba=agefin integra visão AGEFIN

Conexões

↑ recebe de: PDVCaixa (receitas)
 ↑ recebe de: PedidosCompra (CMV)
 ↑ recebe de: SuperAgefin (contas previstas pagas)
 ↑ recebe de: movimentos_caixa
 ↓ envia para: ExtratoConta
 ↓ envia para: ContasFinanceiras (saldo)

Filhas (componentes)

- **ListaLancamentos** — Grelha principal com ações inline
- **FiltrosFluxoCaixa** — Tipo, conta, status, CMV, conciliação
- **ConciliacaoBancaria** — Match LF ↔ extrato bancário
- **NovoLancamentoDialog** — Cria LF manual ou transferência

Regras de negócio

▲ Filtro CMV-only isola custo mercadoria de POs

Funções (lógica server)

◆ gerarExtratoFluxoCaixa [sync (bloqueia UI)] — PDF extrato período

Dados (Supabase)

■ **lancamento_financeiro**
 ■ conta_financeira_id → contas_financeiras.id (FK)
 ■ terceiro_id → terceiro.id
 ■ grupo_lancamento_id → self

● SuperAgefin

/SuperAgefin

pages/SuperAgefin.jsx

Papel

Contas recorrentes e previstas; calendário de compromissos.

Caminho (pathway)

1. conta_recorrente define série
2. Cron gera conta_prevista (3 meses)
3. Operador marca Pago → trigger cria lancamento_financeiro
4. Compromissos sintéticos: sócios (sábados), folha (dia 05)

Conexões

↑ recebe de: PlanejamentoFinanceiro (contas fixas)

↑ recebe de: conta_recorrente

↓ envia para: lancamento_financeiro

↓ envia para: FluxoCaixa

Filhas (componentes)

- SuperAgefinConsultaDrawer — Detalhe conta prevista/recorrente
- SuperAgefinConsultaOrganizer — Organização por mês/categoria

Regras de negócio

- ▲ ContaPrevista Pago → sincronizarContaPrevia cria LF
- ▲ Exclusão recorrente cascata previstas + LF vinculados

Funções (lógica server)

- ◆ cancelarLancamentoFinanceiro [sync (bloqueia UI)] — Reverte saldo conta

Background / performance

- gerarContasPrevistasRecorrentes — cron dia 1
- gerarLancamentosCartao — cron 05:00
- processarLiquidacaoCartaoCredito — cron 08:00

Dados (Supabase)

- **conta_recorrente**
 - terceiro_id → terceiro.id (FK RESTRICT)
 - categoria_financeira_id → categoria_financeira.id (FK RESTRICT)
- **conta_prevista**
 - conta_recorrente_id → conta_recorrente.id (FK SET NULL)

● AprovacoesFinanceiras

/AprovacoesFinanceiras

pages/AprovacoesFinanceiras.jsx + aprovarPedidoCompraFinanceiro.js

Papel

Aprovar/rejeitar pagamentos de pedidos de compra.

Caminho (pathway)

1. Lista POs Aguardando Aprovação Financeira
2. Aprovar → status Aguardando Recepção + LF mantidos
3. Rejeitar → Cancelado + LF cancelados

Conexões

↑ recebe de: PedidosCompra (envio financeiro)

↓ envia para: PedidoCompraDetalhe (recepção)

↓ envia para: ConferenciaEntrada

Filhas (componentes)

- aprovarPedidoCompraFinanceiro.js — Lógica aprovar/rejeitar + LF

Regras de negócio

- ▲ Bloqueia reenvio se parcela LF já paga
- ▲ automacaoAprovacaoFinanceira trigger em status Aprovado

Funções (lógica server)

- ◆ repararLancamentosPedidosAprovados [■ background] — Admin backfill

Dados (Supabase)

- pedido_compra + lancamento_financeiro
- pedido_compra_vinculado_id → pedido_compra.id

● **FolhaPrevisao**

/FolhaPrevisao

pages/FolhaPrevisao.jsx

Papel

Previsão de folha por colaborador e competência.

Árvore desta peça

- FolhaPrevisao
 - FolhaPrevisaoLista
 - FolhaSimulacao

Lógica interna

Alimenta Planejamento e SuperAgefin (compromisso folha dia 05).

Caminho (pathway)

1. Modelo → competência → simulação → grava

Conexões

- ↑ recebe de: folha_centro_custo
- ↓ envia para: PlanejamentoFinanceiro
- ↓ envia para: SuperAgefin

Filhas (componentes)

- **FolhaSimulacao** — Cenários por colaborador

Dados (Supabase)

- **folha_previsao_modelo / folha_previsao_competencia**

● **ContasFinanceiras**

/ContasFinanceiras

financeiro/GestaoContasFinanceiras.jsx

Papel

Cadastro contas bancárias e caixas PDV.

Árvore desta peça

- ContasFinanceiras
 - ListaContasFinanceiras
 - AjusteSaldoDialog

Lógica interna

is_caixa_pdv liga ao SeletorCaixaPDV; saldo derivado de LF.

Caminho (pathway)

- 1. Lista → extrato → ajuste saldo admin

Conexões

- ↑ recebe de: lancamento_financeiro
- ↓ envia para: FluxoCaixa
- ↓ envia para: PDVCaixa

Filhas (componentes)

- ConciliacaoBancaria — Match extrato ↔ LF

Funções (lógica server)

- ◆ auditarSaldosContas [■■ background] — Recomputa saldos

Dados (Supabase)

- contas_financeiras

● VisaoFinanceira

/VisaoFinanceira

config/VisaoFinanceiraPlano.jsx

Papel

Previsto vs realizado consolidado para o dono.

Lógica interna

Agrega conta_prevista + LF + budget numa visão única.

Caminho (pathway)

1. Mês → consolidado → PDF

Conexões

- ↑ recebe de: conta_prevista
- ↑ recebe de: lancamento_financeiro
- ↓ envia para: PlanejamentoFinanceiro

Filhas (componentes)

- planoFinanceiroConsolidado.js — Motor consolidação

Dados (Supabase)

- conta_prevista / lancamento_financeiro

● **CaixasAtivos**

/CaixasAtivos

VisualizadorCaixa.jsx

Papel

Monitor caixas PDV abertos em tempo real.

Lógica interna

Lê turno_caixa Aberto; mostra vendas e sangrias do turno.

Caminho (pathway)

- 1. Lista abertos → drill-down turno

Conexões

- ↑ recebe de: turno_caixa
- ↑ recebe de: consumo_interno
- ↓ envia para: PDVCaixa
- ↓ envia para: TurnosFechados

Filhas (componentes)

- VisualizadorCaixa — Detalhe turno ao vivo

Dados (Supabase)

- turno_caixa / movimentos_caixa

● TurnosFechados

/TurnosFechados

VisualizadorCaixa.jsx (histórico)

Papel

Auditoria de fechamentos por operador/dia.

Caminho (pathway)

1. Filtro data → turnos Fechado → detalhe

Conexões

↑ recebe de: turno_caixa

↓ envia para: Relatorios

↓ envia para: FluxoCaixa

Filhas (componentes)

● CaixaValorDisplay — Formatação valores

Dados (Supabase)

■ turno_caixa

■ movimentos_caixa.turno_caixa_id → turno_caixa.id

● PlanejamentoFinanceiro

/PlanejamentoFinanceiro

planejamento-financeiro-v2/

Papel

Planejamento v2: contas fixas, projeção, cenários.

Árvore desta peça

- PlanejamentoFinanceiro
 - ContasFixasTab
 - ProjecaoTab

Lógica interna

SSOT contas fixas; alimenta SuperAgefin.

Caminho (pathway)

- 1. Modelo → competências → compara LF real

Conexões

- ↑ recebe de: conta_recorrente
- ↑ recebe de: lancamento_financeiro
- ↓ envia para: SuperAgefin
- ↓ envia para: VisaoFinanceira

Filhas (componentes)

- ContasFixasTab — Séries recorrentes

Dados (Supabase)

- budget_modelo / agefin_serie_*

● Budgets

/Budgets

pages/Budgets.jsx

Papel

Orçamento anual por centro de custo.

Caminho (pathway)

1. Modelo → competências mensais → acompanhamento

Conexões

↑ recebe de: [lancamento_financeiro](#)

↓ envia para: [FluxoCaixa](#)

Filhas (componentes)

● **BudgetPlanoCompleto** — Visão anual editável

Dados (Supabase)

■ [budget_modelo](#) / [budget_competencia](#)

Satélites & PWA

Peças acessadas por deep link, share-target ou fluxo lateral — fora do menu principal.

Mapa hierárquico do módulo

Satélites & PWA

- ■ ■ ■ ● AnexoCompartilhado (/AnexoCompartilhado)
- ■ ■ ■ BuscarLancamentoSheet
- ■ ■ ■ AgefinImportador
- ■ ■ ■ ● DevolucaoTroca (/DevolucaoTroca)

● AnexoCompartilhado

/AnexoCompartilhado

pages/AnexoCompartilhado.jsx

Papel

PWA iOS: anexar PDF/boleto a lançamento, PO ou importar pedido.

Árvore desta peça

AnexoCompartilhado

- ■ ■ ■ torre de controlo (destino)
- ■ ■ ■ BuscarLancamentoSheet
- ■ ■ ■ AgefinImportador
- ■ ■ ■ uploadAnexoDrive

Lógica interna

Share-target recebe ficheiro → OCR opcional → utilizador escolhe destino (lançamento existente, novo LF, PO, importar PDF pedido). Ponte entre mundo exterior e financeiro/compras.

Caminho (pathway)

1. 1. iOS share → abre AnexoCompartilhado
2. 2. Escolhe destino (lançamento / PO / importar)
3. 3. uploadAnexoDrive grava anexo_documento
4. 4. Liga referencia_tipo/id à entidade

Conexões

↑ recebe de: iOS share sheet
 ↑ recebe de: OCR PDF
 ↓ envia para: lancamento_financeiro
 ↓ envia para: pedido_compra
 ↓ envia para: SuperAgefin

Filhas (componentes)

- **BuscarLancamentoSheet** — Lista LF de hoje para vincular
- **AgefinImportador** — Importa boleto/recorrente

Regras de negócio

▲ Deep link ?destino=importar_pedido

Funções (lógica server)

- ◆ **uploadAnexoDrive** [sync (bloqueia UI)] — Drive + anexo_documento
- ◆ **listarAnexos** [sync (bloqueia UI)] — Lista por referencia

Dados (Supabase)

- **anexo_documento**
- referencia_tipo/id → polimórfico (LF, PO, conta_prevista)

● DevolucaoTroca

/DevolucaoTroca

pages/DevolucaoTroca.jsx

Papel

Devolução ou troca pós-venda; gera vale_compra.

Lógica interna

Liga devolucao_troca ao pedido_origem; vale debitado no próximo PDVCaixa.

Caminho (pathway)

1. VendasGestao ou PDVCaixa → DevolucaoTroca → vale_compra → próxima venda

Conexões

- ↑ recebe de: pedido_venda
- ↑ recebe de: PDVCaixa
- ↓ envia para: vale_compra
- ↓ envia para: processarVendaCaixa (crédito)

Regras de negócio

▲ substitui_pedido_id liga troca à nova venda

Dados (Supabase)

- devolucao_troca / vale_compra
 - pedido_origem_id → pedido_venda.id
 - cliente_id → terceiro.id

Relatórios

Hub de relatórios; muitos migraram para PDF client-side para evitar lag de deploy.

Mapa hierárquico do módulo

- Relatórios
 - ● Relatorios (/Relatorios)
 - ■■■ Abas por domínio

● Relatorios

/Relatorios

pages/Relatorios.jsx

Papel

Central: Gerencial, Vendas, Compras, Estoque; lança sub-relatórios.

Caminho (pathway)

- 1. Aba → escolhe relatório → PDF client ou server

Conexões

- ↑ recebe de: pedido_venda, pedido_compra, produto, lancamento_financeiro
- ↓ envia para: RelatorioMargem, RelatorioPerformance, PrecoJustoDashboard

Filhas (componentes)

- Abas por domínio — Cada aba agrega links de relatório

Regras de negócio

- ▲ Margem: exclui Cancelado; usa STATUS_PEDIDO_CONTA_NO_TURNO_CAIXA

Funções (lógica server)

- ◆ gerarRelatorioPedidosCompra [sync (bloqueia UI)] — Server PDF
- ◆ gerarRelatorioMargem [sync (bloqueia UI)] — UI prefere client PDF

Dados (Supabase)

- pedido_venda / pedido_compra / produto

Configurações

Admin: utilizadores, perfis, parâmetros globais, ferramentas de manutenção.

Mapa hierárquico do módulo

Configurações

■ ■ ■ ■ ● Configuracoes (/Configuracoes)

■ ■ ■ ■ UsuariosManager

■ ■ ■ ■ PerfisDeAcessoManager

■ ■ ■ ■ AbcdConfigTool

■ ■ ■ ■ MetasEstoqueConfigTool

● Configuracoes

/Configuracoes

pages/Configuracoes.jsx

Papel

Hub com abas Vendas · Operações · Financeiro · Parâmetros · Ferramentas.

Caminho (pathway)

1. Admin abre aba → edita entidade config → propaga para módulos

Conexões

↑ recebe de: perfil_de_acesso (quem pode entrar)

↓ envia para: Todos os módulos (configuracoes_venda, estoque, formas pagamento)

Filhas (componentes)

- **UsuariosManager** — CRUD usuario + p38-auth
- **PerfisDeAcessoManager** — Matriz permissões → menu lateral
- **AbcdConfigTool** — Dispara calcularIEP manual
- **MetasEstoqueConfigTool** — Dispara atualizarMetasEstoque

Regras de negócio

▲ adminOnly no menu; perfilTemEscopoTotal = acesso total

Funções (lógica server)

- ◆ **p38-auth** [sync (bloqueia UI)] — Login, ativar, CRUD users
- ◆ **zerarEntidade** [■ background] — ■ wipe — RecomecarDoZero

Dados (Supabase)

- **usuario**
- perfil_acesso_id → perfil_de_acesso.id
- empresa_id → empresa.id (FK)
- **perfil_de_acesso**

Apêndice — Tabelas-hub

- **terceiro** (CRM) — cliente_id em pedido_venda; fornecedor_id em pedido_compra; terceiro_id em LF
- **produto** (Catálogo) — movimentacao_estoque.produto_id FK RESTRICT; todas as linhas PO/PV
- **pedido_venda** (Vendas) — Origem PDV; alimenta margem, entregas, financeiro
- **pedido_compra** (Compras) — embarque CASCADE; LF is_custo_mercadoria
- **lançamento_financeiro** (Financeiro) — referencia polimórfica; saldo em contas_financeiras
- **turno_caixa** (Sessão) — movimentos_caixa FK; agrupa vendas do operador